Optimization Techniques in Deep Learning

STAT 525 - Topics in Computational Statistics

Shulei Wang

University of Illinois at Urbana-Champaign

shuleiw@illinois.edu

https://sites.google.com/view/shulei-wang/



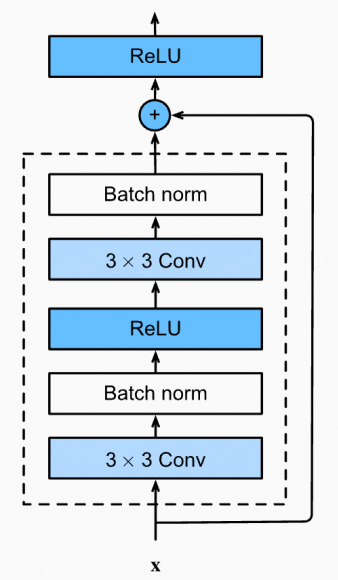
Overview

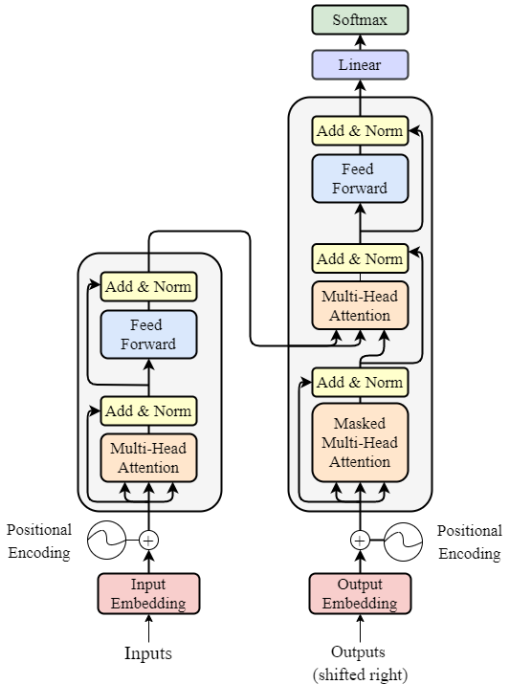
Back-Propagation

Optimization for Deep Learning

Adaptive Learning Rate/Step Size

Deep Neural Network





- Deep neural network is one the most powerful tools in modern machine learning.
- Many successful examples: AlphaGo, AlphaFold, and ChatGPT.

Deep Neural Network

- Deep neural network is a compositional nonlinear map. The transformation at layer l

$$X_i^l = \sigma\left(W^l X_i^{l-1} + g^l\right),$$

where W^l is a matrix, g^l is a vector, and σ is an activation function, including
  - Sigmoid: $\sigma(t) = 1/(1+e^{-t})$
  - Rectified Linear Unit: $\sigma(t) = \max(t, 0)$.
- Sometimes, the architecture of deep neural network can be complicated, e.g., ResNet and transformer.

Deep Neural Network

- In classification, softmax is applied to the last layer of deep neural networks so we can have "probability" as output.
- Cross entropy loss is usually used

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^{n} L(X_i, Y_i, \beta),$$

where

$$L(X_i, Y_i, \beta) = \sum_{k=1}^{K} Y_{i,k} \psi_\beta(X_i)_k - \log \left(\sum_{k=1}^{K} \exp(\psi_\beta(X_i)_k) \right).$$

Here, ψ_β is deep neural network with parameter β.

- The first step is to evaluate the gradient of $L(X_i, Y_i, \beta)$ if we would like to solve the above optimization problem.

Nested Composition Function

- We first consider the nested composition function $f : \mathbb{R}^n \to \mathbb{R}$

$$\begin{aligned} f(x) &= \phi(\phi_l(\phi_{l-1}(\ldots(\phi_1(x))\ldots))) \\ &= (\phi \circ \phi_l \circ \phi_{l-1} \circ \ldots \circ \phi_1)(x), \end{aligned}$$

  where $m_0 = n$ and

$$\phi_i : \mathbb{R}^{m_{i-1}} \to \mathbb{R}^{m_i},\ i = 1, \ldots, l \quad \text{and} \quad \phi : \mathbb{R}^{m_l} \to \mathbb{R}.$$

- What is the gradient of $f(x)$?

Nested Composition Function

- By chain rule, the gradient of $f(x)$ can be written as

$$\nabla f(x) = (\nabla_x \phi_1)(\nabla_{\phi_1} \phi_2) \dots (\nabla_{\phi_{l-1}} \phi_l)(\nabla_{\phi_l} \phi),$$

  where
  - $\nabla_x \phi_1$ is a matrix of dimension $n \times m_1$
  - $\nabla_{\phi_i} \phi_{i+1}$ is a matrix of dimension $m_i \times m_{i+1}$
  - $\nabla_{\phi_l} \phi$ is a vector of length m_l
- Here $\nabla_{\phi_i} \phi_{i+1}$ is the gradient of ϕ_{i+1} at $(\phi_i \circ \dots \circ \phi_1)(x)$.
- How shall we evaluate $\nabla f(x)$ efficiently?

Nested Composition Function

Here is an algorithm for gradient evaluation:

1. Given x, evaluate $x_1 = \phi_1(x)$ and $A_1 = \nabla_x \phi_1(x)$.
2. for $i = 1, 2, \ldots, l-1$
   - evaluate

$$x_{i+1} = \phi_{i+1}(x_i) \qquad \text{and} \qquad A_{i+1} = \nabla_{\phi_i} \phi_{i+1}(x_i)$$

3. Evaluate $f = \phi(x_l)$ and $p_l = \nabla_{\phi_l} \phi(x_l)$.
4. for $i = l, l-1, \ldots, 2$
   - evaluate

$$p_{i-1} = A_i p_i$$

5. Define $g = A_1 p_1$.
6. Output $f = f(x)$ and $g = \nabla f(x)$.

Nested Composition Function

- Two evaluation loops
  - Forward pass: evaluate the function and store partial derivative
  - Reverse pass: accumulate partial derivative
- The algorithm is efficient as it minimum number of matrix/vector multiplication.
- Deep neural network is not a nested composition function with respect to the parameters. The parameters of each layer are introduced progressively.

Progressive Function

- We consider a progressive function

$$f(x) = \phi(\phi_l(x_l, \phi_{l-1}(x_{l-1}, \dots (x_2, \phi_1(x_1)) \dots))),$$

where $x = (x_1, \dots, x_l)$ with $x_i \in \mathbb{R}^{n_i}$ such that $n = \sum_{i=1}^{l} n_i$ and

$\phi_1 : \mathbb{R}^{n_1} \to \mathbb{R}^{m_1}, \phi_i : \mathbb{R}^{n_i} \times \mathbb{R}^{m_{i-1}} \to \mathbb{R}^{m_i}$, $2 \leq i \leq l$ and $\phi : \mathbb{R}^{m_l} \to \mathbb{R}$.

Here, the input of ϕ_i is x_i and the output of ϕ_{i-1}.

- In sequential deep neural network, x_i is the weights in the ith layer, ϕ_i is the output in the ith layer of one sample, and ϕ is the loss function of one sample.

Progressive Function

- How does the gradient of progressive function look like?

$$
\begin{aligned}
\nabla_{x_l} f(x) &= (\nabla_{x_l} \phi_l)(\nabla_{\phi_l} \phi) \\
\nabla_{x_{l-1}} f(x) &= (\nabla_{x_{l-1}} \phi_{l-1})(\nabla_{\phi_{l-1}} \phi_l)(\nabla_{\phi_l} \phi) \\
\nabla_{x_{l-2}} f(x) &= (\nabla_{x_{l-2}} \phi_{l-2})(\nabla_{\phi_{l-2}} \phi_{l-1})(\nabla_{\phi_{l-1}} \phi_l)(\nabla_{\phi_l} \phi) \\
&\dots \\
\nabla_{x_1} f(x) &= (\nabla_{x_1} \phi_1)(\nabla_{\phi_1} \phi_2) \dots (\nabla_{\phi_{l-2}} \phi_{l-1})(\nabla_{\phi_{l-1}} \phi_l)(\nabla_{\phi_l} \phi)
\end{aligned}
$$

- The gradient has a strong pattern as they share a lot of similar structure

$$
\nabla_{x_i} f(x) = (\nabla_{x_i} \phi_i)(\nabla_{\phi_i} \phi_{i+1}) \dots (\nabla_{\phi_{l-2}} \phi_{l-1})(\nabla_{\phi_{l-1}} \phi_l)(\nabla_{\phi_l} \phi)
$$

- How can we evaluate gradient efficiently?

Progressive Function

Here is an algorithm for gradient evaluation:

1. Given $x = (x_1, \ldots, x_l)$, evaluate $s_1 = \phi_1(x_1)$ and $B_1 = \nabla_{x_1}\phi_1(x_1)$.
2. for $i = 1, 2, \ldots, l-1$
   - evaluate

$$s_{i+1} = \phi_{i+1}(x_{i+1}, s_i), \qquad A_{i+1} = \nabla_{\phi_i}\phi_{i+1}(x_{i+1}, s_i)$$

$$B_{i+1} = \nabla_{x_{i+1}}\phi_{i+1}(x_{i+1}, s_i)$$

3. Evaluate $f = \phi(s_l)$ and $p_l = \nabla_{\phi_l}\phi(s_l)$.
4. for $i = l, l-1, \ldots, 2$
   - evaluate

$$p_{i-1} = A_i p_i, \qquad g_i = B_i p_i$$

5. Define $g_1 = B_1 p_1$.
6. Output $f = f(x)$ and $g = (g_1, \ldots, g_l) = \nabla f(x)$.

Progressive Function

- The same two evaluation loops
  - Forward pass: evaluate the function and store partial derivative
  - Reverse pass: accumulate partial derivative
- The algorithm is still efficient. The computational cost of finding gradient is just a multiply of that of evaluating function.
- The deep neural network in practice can be much more complex than sequential deep neural network.

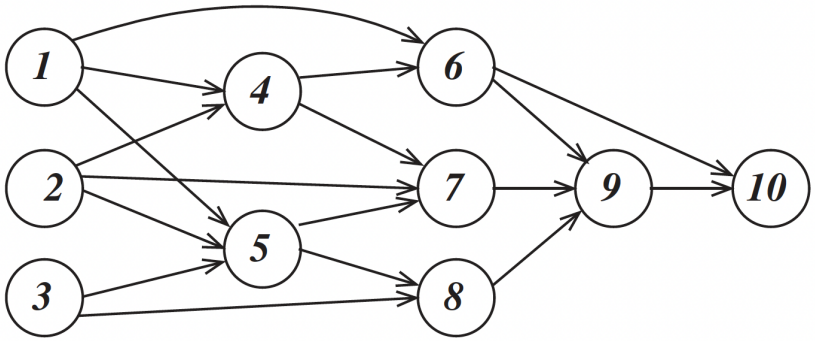
Automatic Differentiation

- The current stage may not only depend on previous stage, but also on many prior stages, such as ResNet.
- We express such a function by a directed acyclic graph (DAG), called computation graph. Besides $x = (x_1, \ldots, x_l)$, we introduce $x_{l+1}, \ldots, x_L$ and they can be computed in the following way

$$x_i = \phi_i(x_{\mathcal{P}(i)}), \qquad i = l+1, \ldots, L,$$

  where $\mathcal{P}(i)$ denotes the parents of node i.
- We can usually assume $\mathcal{P}(i) \subset \{1, 2, \ldots, i-1\}$.

Automatic Differentiation



- Above figure is an example of computation graph

$$\mathcal{P}(4) = \{1, 2\},\ \mathcal{P}(5) = \{1, 2, 3\},\ \mathcal{P}(6) = \{1, 4\},\ \mathcal{P}(7) = \{2, 4, 5\},$$
$$\mathcal{P}(8) = \{3, 5\},\ \mathcal{P}(9) = \{6, 7, 8\},\ \mathcal{P}(10) = \{6, 9\}.$$

Automatic Differentiation

Here is an algorithm for gradient evaluation:

1. Given $x = (x_1, \ldots, x_l)$.
2. for $i = l+1, l+2, \ldots, L$
   - evaluate

$$x_i = \phi_i(x_{\mathcal{P}(i)}), \qquad \frac{\partial x_i}{\partial x_j} = \nabla_{x_j} \phi_i(x_{\mathcal{P}(i)}), \quad j \in \mathcal{P}(i).$$

3. Introduce $z_i = 0$ for $i = 1, \ldots, L-1$ and $z_L = 1$.
4. for $i = L, L-1, \ldots, 1$
   - evaluate

$$z_j = z_j + z_i \frac{\partial x_i}{\partial x_j}, \qquad j \in \mathcal{P}(i)$$

5. Output $x_L = f(x)$ and $g = (z_1, \ldots, z_l) = \nabla f(x)$.

Automatic Differentiation

- This algorithm is reverse mode of automatic differentiation or algorithmic differentiation, also called back-propagation in machine learning.
- The computational cost of finding gradient is bounded by a small multiple of the cost of evaluating function.
- The main issue of reverse mode is the space complexity. We need to store (x_i, z_i) for $i = 1, \ldots L$ and $\partial x_i / \partial x_j$ for each edge in computation graph. The space complexity grows linearly with the time complexity of evaluating f.
- There is also a forward mode of automatic differentiation, which is more efficient when the dimension of input is much smaller than the dimension of output.

Overview

Back-Propagation

Optimization for Deep Learning

Adaptive Learning Rate/Step Size

Optimization Problem in Deep Learning

- Recall that we aim to optimize the following problem

$$\min_{\beta} \frac{1}{n} \sum_{i=1}^{n} L(X_i, Y_i, \beta),$$

  and know how to compute $\nabla_\beta L(X_i, Y_i, \beta)$ by back-propagation.
- The objective function is non-convex
  - Potentially many local minima
  - Saddle points
- Despite the non-convexity, we can still apply optimization technique we learned before.

Initialization in Non-Convex Optimization

- In convex problem, the convergence is guaranteed no matter what initial point is.
- In the non-convex optimization problem, initialization is much more important.
- Deep neural networks are not well understood to have principled,mathematically nice initialization strategies.
  - What is known: Initialization should break symmetry
  - What is known: Scale of weights is important
- Most initialization strategies are based on intuitions and heuristics

Initialization in Deep Learning

Two heuristics for deep learning

- For a fully connected layer with m inputs and n outputs, our initial weights can be drawn from uniform distribution:

$$W_{i,j} = U\left(-\frac{1}{\sqrt{m}}, \frac{1}{\sqrt{m}}\right).$$

- Xavier Initialization:

$$W_{i,j} = U\left(-\frac{6}{\sqrt{m+n}}, \frac{6}{\sqrt{m+n}}\right).$$

- They work well in practice.

- The first method we could try is gradient descent

$$\beta^{(k+1)} = \beta^{(k)} - t^{(k)} \frac{1}{n} \sum_{i=1}^{n} \nabla_\beta L(X_i, Y_i, \beta^{(k)})$$

  - Pro: gradient estimate is stable
  - Con: need to compute the gradient on each sample in one update

Optimization Problem in Deep Learning

- In practice, people may prefer stochastic gradient descent due the large sample size

$$\beta^{(k+1)} = \beta^{(k)} - t^{(k)} \nabla_\beta L(X_{i^{(k)}}, Y_{i^{(k)}}, \beta^{(k)})$$

  - Pro: need to compute the gradient on one sample in one
  - Con: gradient estimate can be very noisy and decaying step size
- To mitigate the issue, we can also consider stochastic gradient descent on minibatch

$$\beta^{(k+1)} = \beta^{(k)} - t^{(k)} \frac{1}{|S^{(k)}|} \sum_{i \in S^{(k)}} \nabla_\beta L(X_i, Y_i, \beta^{(k)})$$

Optimization Problem in Deep Learning

- Another common strategy to stabilize stochastic gradient descent is momentum

$$\beta^{(k+1)} = \beta^{(k)} - t^{(k)} v^{(k)},$$

where $v^{(k)}$ is the velocity

$$v^{(k)} = \alpha v^{(k-1)} + \nabla_\beta L(X_{i^{(k)}}, Y_{i^{(k)}}, \beta^{(k)}).$$

- Similarly, we have Nesterov momentum

$$v^{(k)} = \alpha v^{(k-1)} + \nabla_\beta L(X_{i^{(k)}}, Y_{i^{(k)}}, \beta^{(k)} - \alpha v^{(k-1)}).$$

- In all methods we learned, we use a universal step size for all features in β. If different features very in importance, can we design the step size for each individual feature?

Overview

Back-Propagation

Optimization for Deep Learning

Adaptive Learning Rate/Step Size

Adaptive Step Size

- The idea in adaptive step size is to assign step size to each feature

$$\beta_j^{(k+1)} = \beta_j^{(k)} - t_j^{(k)} \nabla_{\beta_j} L(X_{i^{(k)}}, Y_{i^{(k)}}, \beta^{(k)}),$$

  where $t_j^{(k)}$ is the step size designed for the feature j.
- But how to choose $t_j^{(k)}$?

AdaGrad

- AdaGrad is one of the first adaptive methods.
- The idea is to downscale the step size by square-root of sum of squares of all its historical values.

$$t_j^{(k)} = \frac{\eta}{\sqrt{\sum_{l=0}^{k} \nabla_{\beta_j} L(X_{i^{(l)}}, Y_{i^{(l)}}, \beta^{(l)})^2 + \epsilon}}$$

- Features that have large gradient variance $\Rightarrow$ step sizes for them are rapidly declined.
- Duchi, J., Hazan, E. and Singer, Y., 2011. Adaptive subgradient methods for online learning and stochastic optimization. Journal of machine learning research, 12(7).

AdaGrad

Here is the algorithm for AdaGrad:

1. Initialize $r = 0 \in \mathbb{R}^d$.
2. for $k = 1, 2, \ldots$
   - Randomly choose one sample $(X_{i^{(k)}}, Y_{i^{(k)}})$.
   - Evaluate the gradient $g^{(k)} = \nabla_\beta L(X_{i^{(k)}}, Y_{i^{(k)}}, \beta^{(k)})$
   - Accumulate second moment estimate $r_j = r_j + (g_j^{(k)})^2$
   - Update

$$\beta_j^{(k+1)} = \beta_j^{(k)} - \frac{\eta}{\sqrt{r_j} + \epsilon} g_j^{(k)}, \qquad j = 1, \ldots, d.$$

RMSprop

- AdaGrad is designed for convex problem. It might shrink the step size too aggressively and the step size becomes smaller and smaller due to the monotonic increment of the running squared sum.
- We can adapt it to perform better in non-convex settings by accumulating an exponentially decaying average of the gradient.
- The method is called RMSprop (Root Mean Squared Propagation), which is first proposed by Geoff Hinton in lecture 6 of the online course "Neural Networks for Machine Learning".

RMSprop

Here is the algorithm for RMSprop:

1. Initialize $r = 0 \in \mathbb{R}^d$.
2. for $k = 1, 2, \ldots$
   - Randomly choose one sample $(X_{i^{(k)}}, Y_{i^{(k)}})$.
   - Evaluate the gradient $g^{(k)} = \nabla_\beta L(X_{i^{(k)}}, Y_{i^{(k)}}, \beta^{(k)})$
   - Accumulate second moment estimate $r_j = \rho r_j + (1-\rho)(g_j^{(k)})^2$
   - Update

$$\beta_j^{(k+1)} = \beta_j^{(k)} - \frac{\eta}{\sqrt{r_j} + \epsilon} g_j^{(k)}, \qquad j = 1, \ldots, d.$$

Adam

- RMSprop can also work with the idea in momentum.
- Adam is like RMSProp with Momentum but with bias correction terms for the first and second moments.
- Adam may be the first optimizer you want to try in training of deep neutral networks.
- Kingma, D.P. and Ba, J., 2014. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980

Adam

Here is the algorithm for Adam:

1. Initialize $r, v = 0 \in \mathbb{R}^d$.
2. for $k = 1, 2, \ldots$
   - Randomly choose one sample $(X_{i^{(k)}}, Y_{i^{(k)}})$.
   - Evaluate the gradient $g^{(k)} = \nabla_\beta L(X_{i^{(k)}}, Y_{i^{(k)}}, \beta^{(k)})$
   - Update velocity $v = \alpha v + (1 - \alpha) g^{(k)}$.
   - Accumulate second moment estimate $r_j = \rho r_j + (1 - \rho)(g_j^{(k)})^2$
   - Update

$$\beta_j^{(k+1)} = \beta_j^{(k)} - \frac{\eta}{\sqrt{r_j/(1-\rho^k)} + \epsilon} v_j/(1-\alpha^k), \qquad j = 1, \ldots, d.$$